MONASH University
Information Technology

MONASH University
Information Technology

FIT3080 – Artificial Intelligence

Solving Problems as Search – Informed Search

Sections 3.5.1-3.5.5, 3.6.1, 3.6.2

LN3 Learning Objectives

- **Problem formulation**
- **Problem-solving agents**
- **Search algorithms and strategies**
  - **Tentative algorithms**
    - > Backtrack [Chapter 5.1, 5.3, 5.4]
    - > Tree and graph search **strategies**:
      - **Uninformed:** BFS, UCS, DFS, DLS, IDS [Chapter 3.1-3.4.4, 3.4.6]
      - **Informed**: Greedy best-first search, A, A* [Chapter 3.5.1-3.5.4]
  - **Irrevocable algorithms**
    - > **Informed**: Hill climbing, Local beam search, Simulated annealing, Genetic algorithms [Chapter 4.1]
- **Adversarial search algorithms [Chapter 6.1-6.3]**
  - Optimal decisions
  - Minimax, α-β pruning

Heuristic (Informed) Graphsearch Procedures

- **Problem with Uniform Cost Search**
  - It doesn't know where the goal is
- **Use *Heuristic Information* (domain dependent information) to help reduce the search**
  - Evaluation function – a real valued function used to compute the "promise" of a node

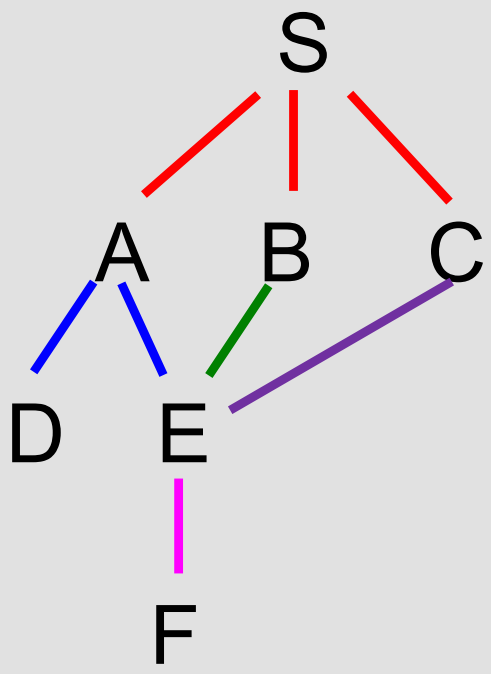
Best-First Search Algorithm

function BESTFIRSTSEARCH(*problem*) **returns** a solution or failure

- Initialize the *frontier* using the initial state of *problem*
- Initialize the *explored set* to ∅
- **Loop**
  1. **if** the *frontier* is ∅ **then return** failure
  2. ***Get the first*** leaf node and remove it from the *frontier*
  3. **if** the node contains a goal state **then return** the corresponding solution
  4. add the node to the *explored set*
  5. ***expand*** the chosen node → generate its children
     a. **if** the resulting nodes are not in the *frontier* or *explored* **then add** them to the *frontier*
     b. **else if** the resulting nodes are in the *frontier* or in *explored* ***with a higher path cost*** **then *merge*** them with these nodes and update their path

End

Adding and Merging Nodes



Heuristic Graphsearch: Definitions (I)

- $k(n_i,n_j)$ – actual cost of minimal cost path between n_i and n_j
- $h^*(n) = \min\{k(n,t_i)\}$
  minimum of all the $k(n,t_i)$ over the entire set of goal nodes $\{t_i\}$
- $g^*(n) = k(s,n)$
  minimum cost from the start node s to n
- $f^*(n)=g^*(n)+h^*(n)$
  cost of an optimal path constrained to go through n
- $f^*(s)=h^*(s)$
  cost of an unconstrained optimal path from s to goal

Heuristic Graphsearch: Definitions (II)

- **$f(n)$ – *estimate* of the minimal cost path constrained to go through node n**
- **$g(n)$ – *estimate* of $g^*(n)$ ($g(n) \geq 0$)**
  - Usual choice: Cost of the path in the search tree/graph from s to n ➔ $g(n) \geq g^*(n)$
- **$h(n)$ – heuristic function**
  Estimate of $h^*(n)$ ($h(n) \geq 0$)

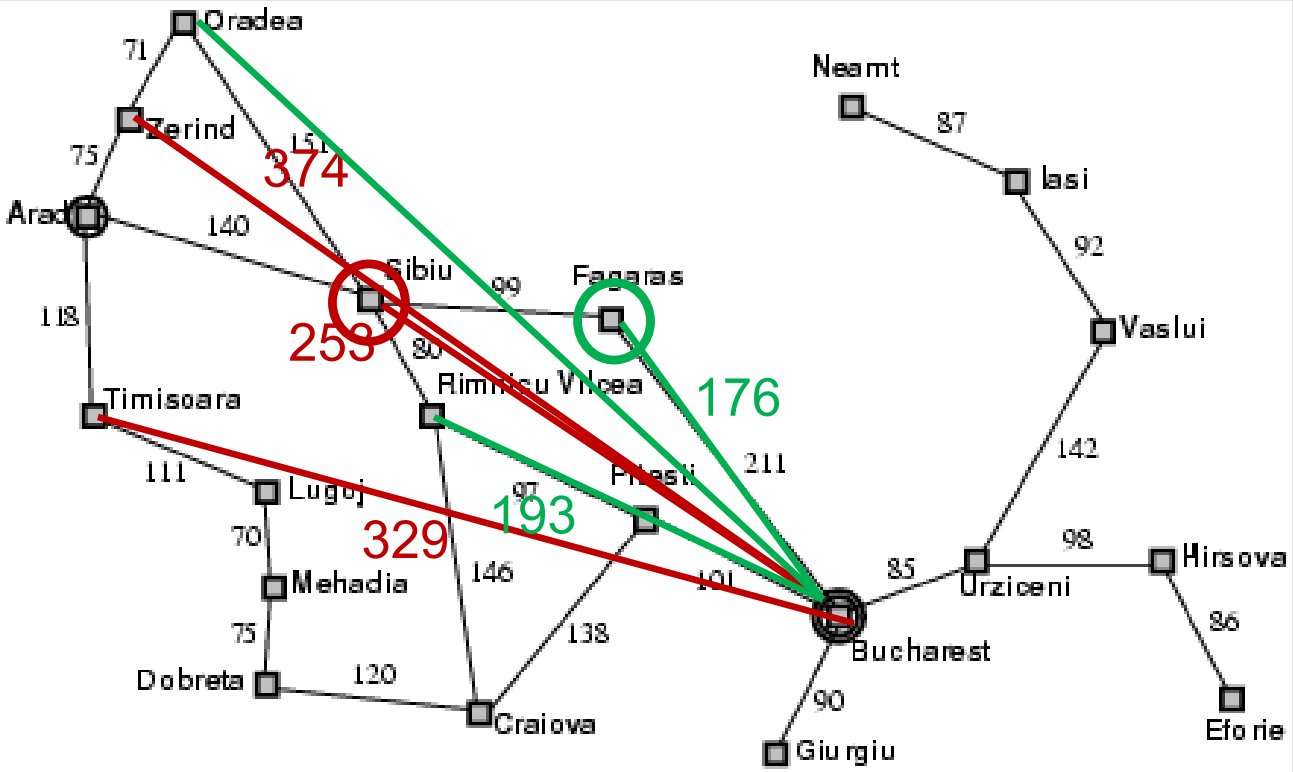
MONASH University
Information Technology

FIT3080 – Artificial Intelligence

Greedy Best-first Search

Greedy Best-First Search

- **Expands the node that is closest to the goal among the *current options***
  - $f(n) = h(n)$
  - Example: $h_{SLD}(n)$ = Straight-Line Distance to the goal



Properties of Greedy Best-First Search

- **Complete?**
  - Infinite-state spaces: No
  - Finite-state spaces: Yes, if we check for ancestors
- **Time?** $O(b^m)$
- **Space?** $O(b^m)$
- **Optimal?** **No**

MONASH University
Information Technology

FIT3080 – Artificial Intelligence

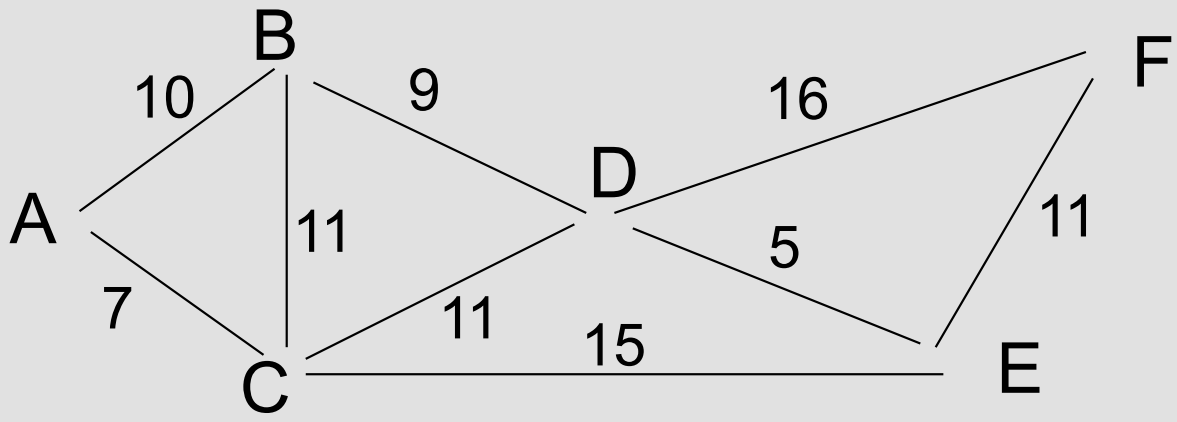
Algorithm A and A*

Algorithm A and A*

- **Graphsearch uses the evaluation function *f(n) = g(n) + h(n)***
  - $g(n) \geq g^*(n)$
  - $h(n) \geq 0$ *(algorithm A)*
  - $h(n) \leq h^*(n)$ *(algorithm A*)*
- **Expands next the node in the frontier with the smallest value of *f(n)***

- Hart, P.E., Nilsson, N.J. and Raphael, B., 1968. A formal basis for the heuristic determination of minimum cost paths. IEEE transactions on Systems Science and Cybernetics, 4(2), pp.100-107.
- Nils Nilsson: https://www.youtube.com/watch?v=AZuTTODTtvo

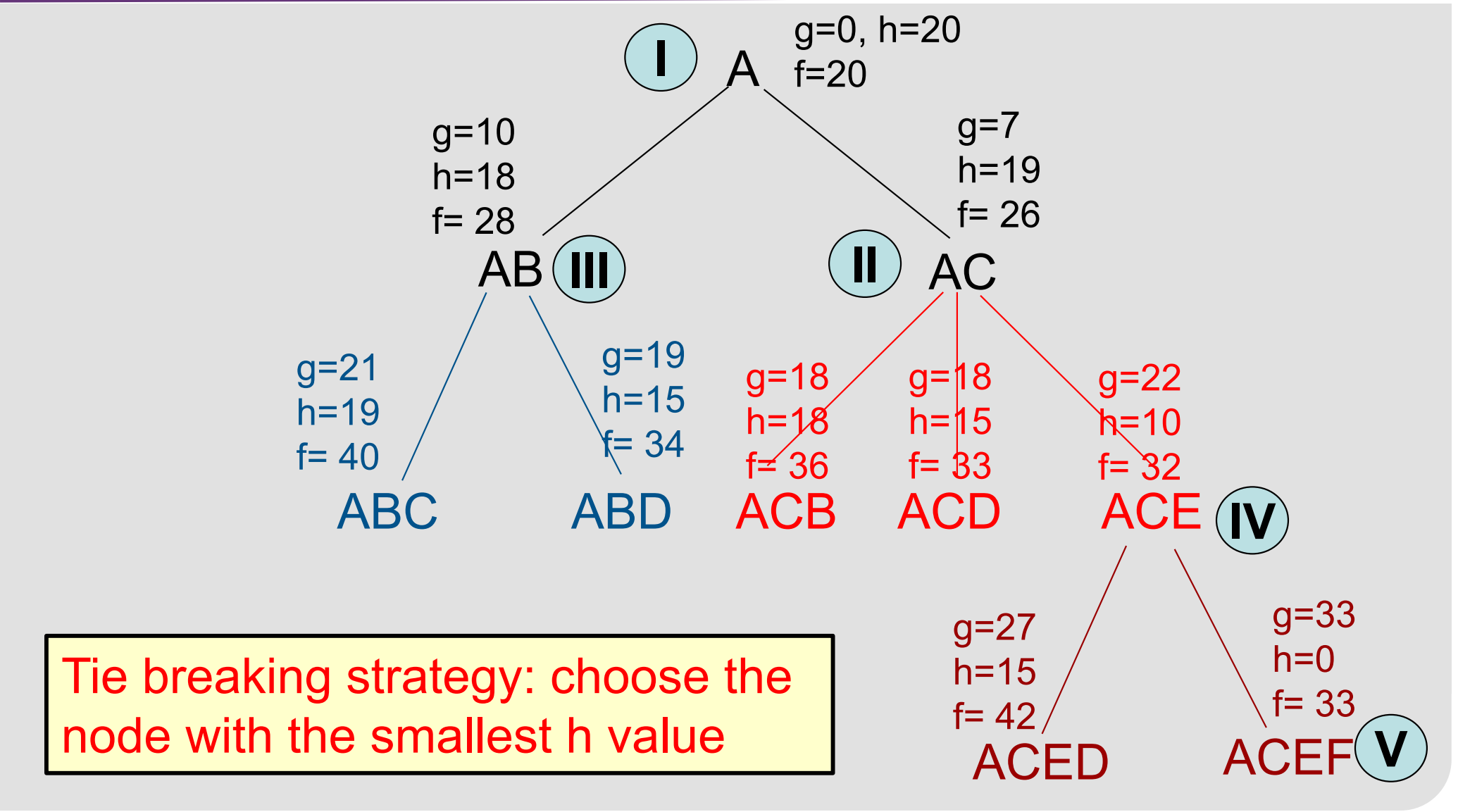
Algorithm A* Example – Shortest Path (I)



ROAD DISTANCES						
	A	B	C	D	E	F
A		10	7			
B			11	9		
C				11	15	
D					5	16
E						11

AIR DISTANCES						
	A	B	C	D	E	F
A		4	3	8	12	20
B			6	5	9	18
C				7	10	19
D					5	15
E						10

Algorithm A* Example – Shortest Path (II)



Admissibility and Consistency/Monotonicity

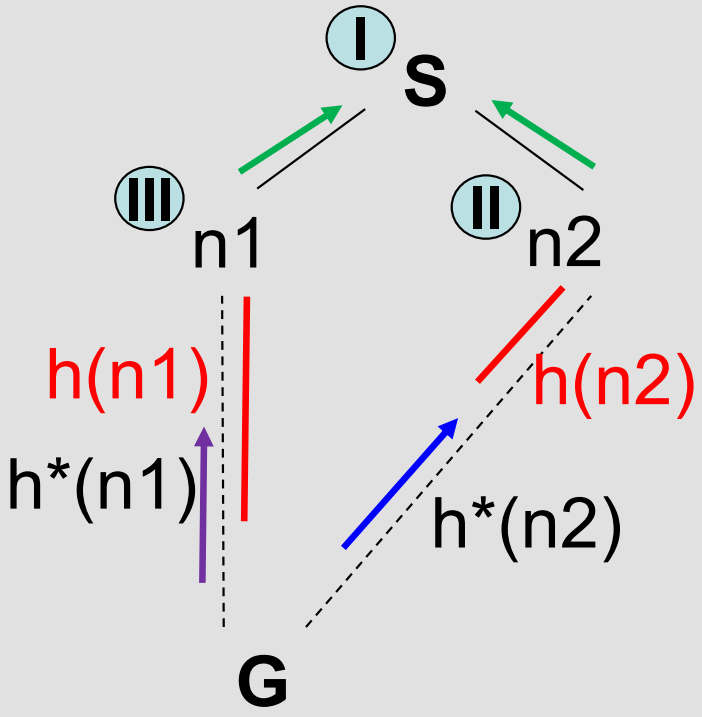
- ***Admissibility* of h:**
  If $\forall n \;\; h(n) \leq h^*(n)$
  Then A* is guaranteed to find the optimal solution (if it exists)
- ***Consistency (Monotonicity)* of h:**
  If $\forall n \;\; h(n) \leq c(n,m) + h(m)$, where m is a child of n
  Then A* has found the optimal path to any node it selects for expansion
- **Consistent heuristics are admissible**

Which uninformed search algorithms can be considered Algorithm A with consistent *h*?

h(n) – Admissibility

If $\forall n \;\; h(n) \leq h^*(n)$

Then A* is guaranteed to find the optimal solution (if it exists)



g(n1) = g(n2), h(n2) < h(n1)

frontier = {n2,n1}

n2 is expanded: g(G)=g(n2)+h*(n2)

If h(n1) ≥ h*(n2) then

frontier = {G,n1}, G is chosen

Else

frontier = {n1,G}, n1 expanded:

g(G)=g(n1)+h*(n1)

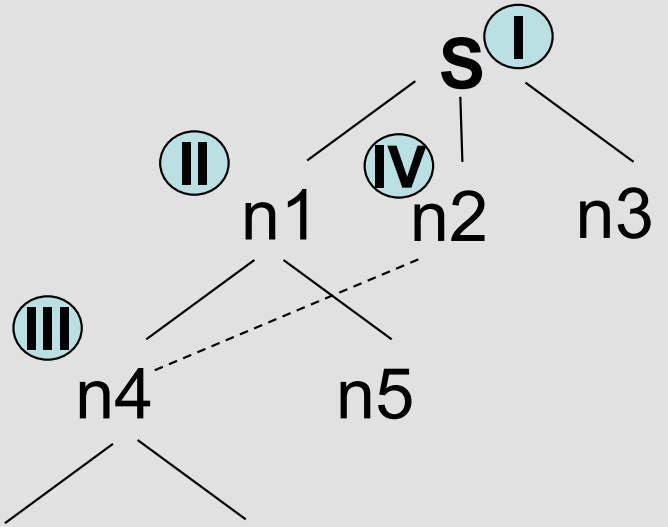
recalculate f(G)

min{ g(n1)+h*(n1), g(n2)+h*(n2) }

h(n) – Consistency / Monotonicity

If $\forall n \quad h(n) \leq c(n,m) + h(m)$, where m is a child of n

Then A* has found the optimal path to any node it selects for expansion



f(n1) ≤ f(n_i) for i=2,3 ➔ n1 is expanded

f(n4) ≤ f(n_i) for i=2,3,5 ➔ n4 is expanded

n2 is expanded: do we recalculate f(n4)?

f(n4) = g(n4) +h(n4) ≤ g(n2)+h(n2) = f(n2)

= g(n1)+c(n1,n4) +h(n4) ≤ g(n2)+h(n2) ≤ g(n2)+c(n2,n4)+h(n4)

monotonicity

n4 through n1

n4 through n2

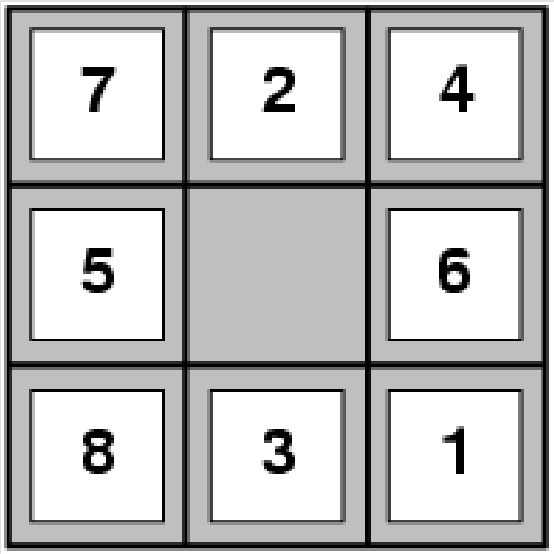
Properties of A and A*

	A	A*
Complete?	Yes	Yes
Time?	$O(b^m)$	$O(b^\Delta)$, where $\Delta \alpha \max\|h-h^*\|$
Space?	$O(b^m)$	$O(b^\Delta)$
Optimal?	No	Yes

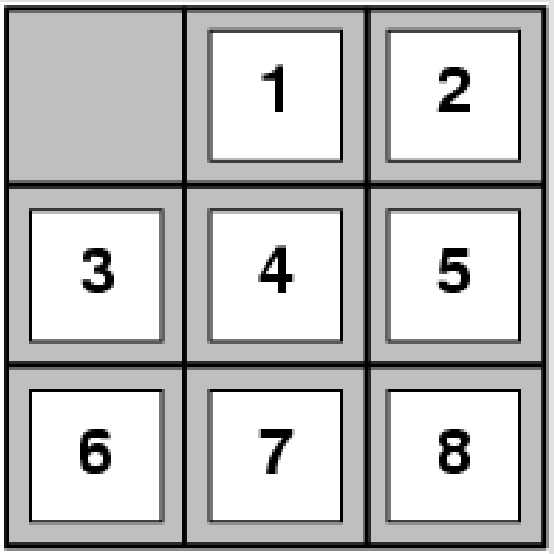
Admissible heuristics: 8 Puzzle

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total *Manhattan distance* (# of squares from desired location of each tile)

- $h_1(S) = ?$
- $h_2(S) = ?$



Start State



Goal State

Relaxed problems

- **A problem with fewer restrictions on the actions is called a *relaxed problem***
- **The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem**
- **Examples:**
  - If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, then $h_1(n)$ gives the shortest solution
  - If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution

Dominance

- **Given two admissible heuristics h_1 and h_2, if $h_2(n) \geq h_1(n)$ for all n then h_2 *dominates* h_1**

  → h_2 is better for search

- **If we have several admissible heuristics h_1, h_2, ..., h_n, none of which dominates, we can take the maximum:**

$$h(i) = \max\{h_1(i), h_2(i), ..., h_n(i)\}$$

The Effect of Heuristics on Performance

The quality of a heuristic can be characterized by the *effective branching factor (EBF)* b^*

- **If N nodes are generated, this is the branching factor that a uniform tree of depth d should have to contain $N+1$ nodes:**

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d = \frac{(b^*)^{d+1} - 1}{b^* - 1}$$

Approximating with $b^* \approx 2$

$$N + 1 = (b^*)^{d+1} - 1 \rightarrow \sqrt[d+1]{N+2} = b^*$$

Examples with $b^* \approx 2$ and different values of *N*:

- $N=52, d=5 \rightarrow b^* \approx 1.92$
- $N=20, d=5 \rightarrow b^* \approx 1.67$

→ Experimental measurements of b^* on a small set of problems can provide an idea of a heuristic's usefulness

- A good heuristic yields $b^* \approx 1$

MONASH University
Information Technology

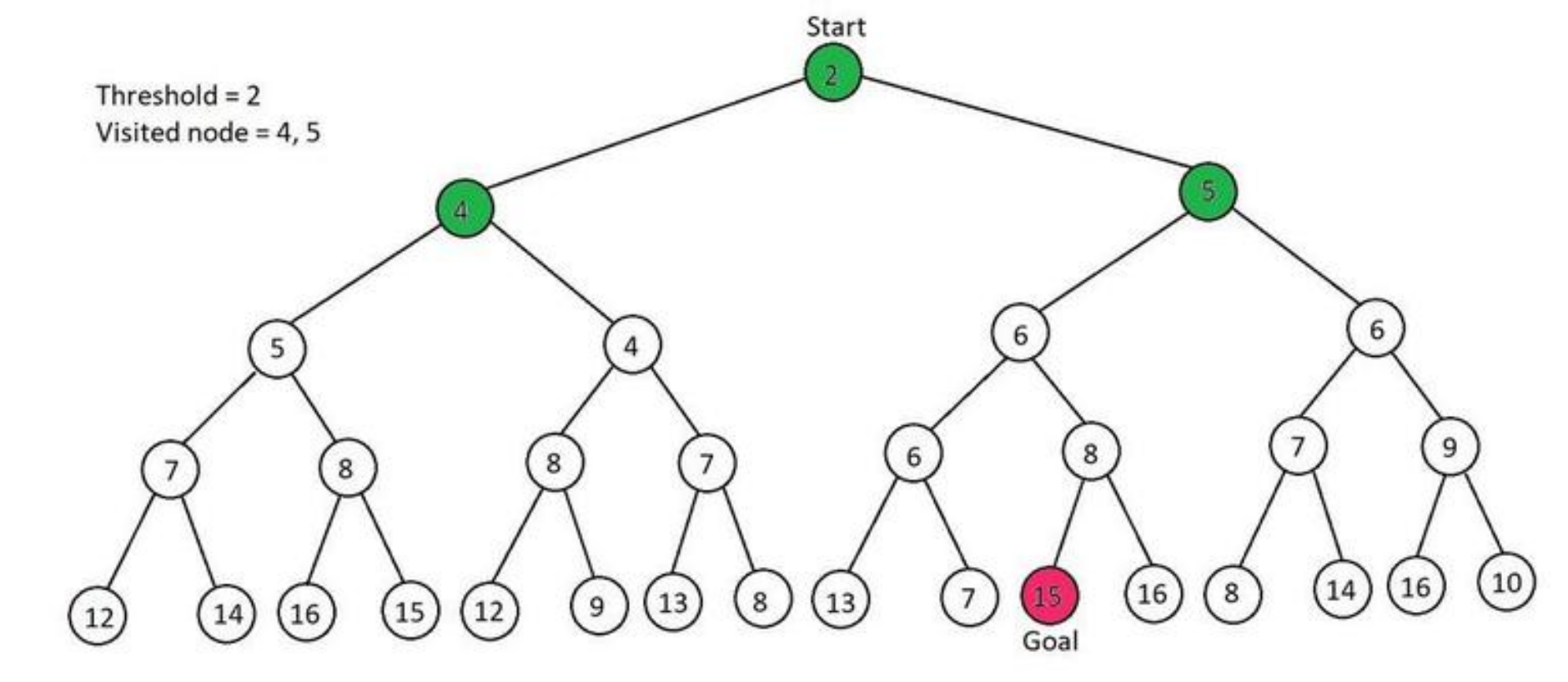
FIT3080 – Artificial Intelligence

Variants of A*

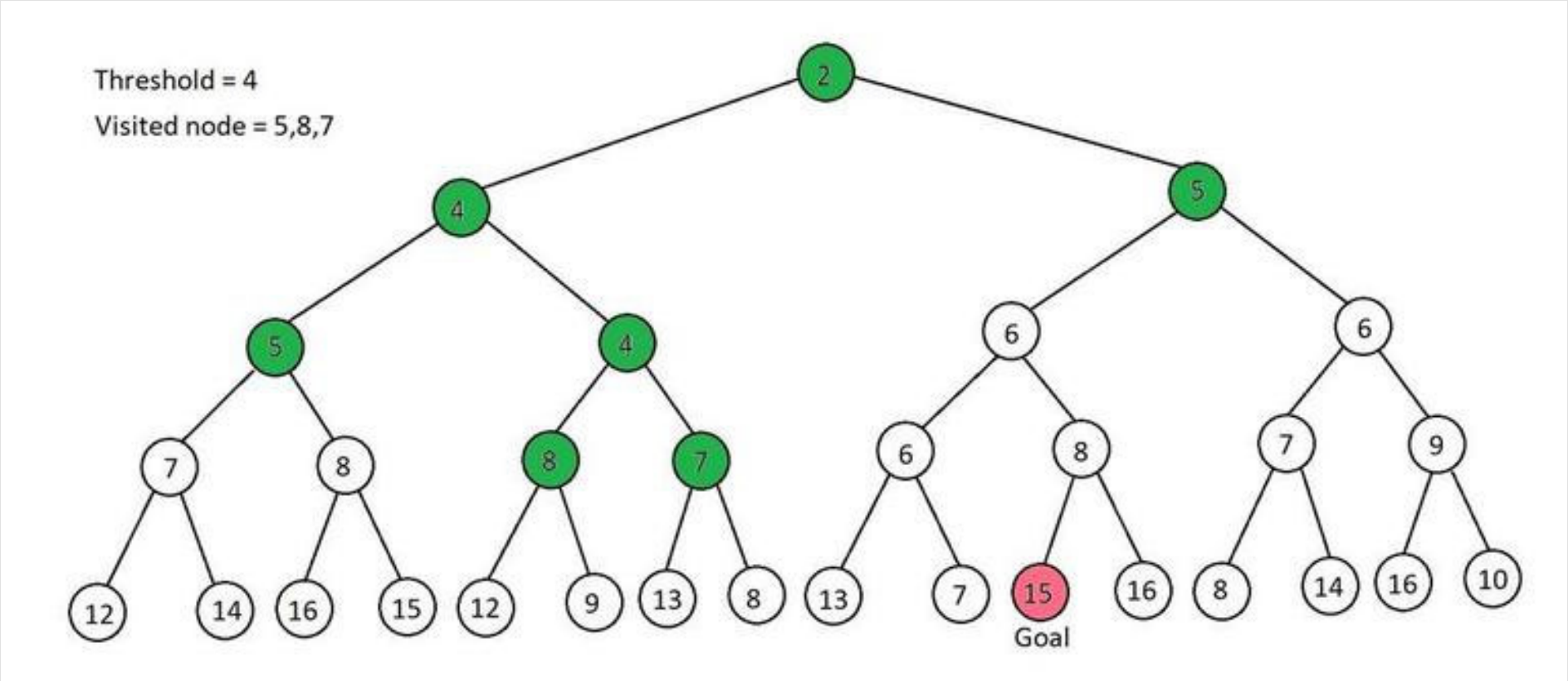
Iterative Deepening A* (IDA*)

- The Tree-Search version of A* can be effectively combined with iterative deepening
  - Similar to ID-DFS but now we search with f-costs.
  - There is a cost limit (instead of a depth limit)
- IDA* sketch
  1. Initially $f_{lim} = h(start)$
  2. For each generated node n, compute $f(n) = g(n) + h(n)$
  3. Expand all nodes with $f(n) \leq f_{lim}$
  4. Update f_{lim} = the smallest f-cost of any node that exceeded the cutoff on the previous iteration
  5. Repeat 2-4 until the goal is **considered** or the tree is exhausted

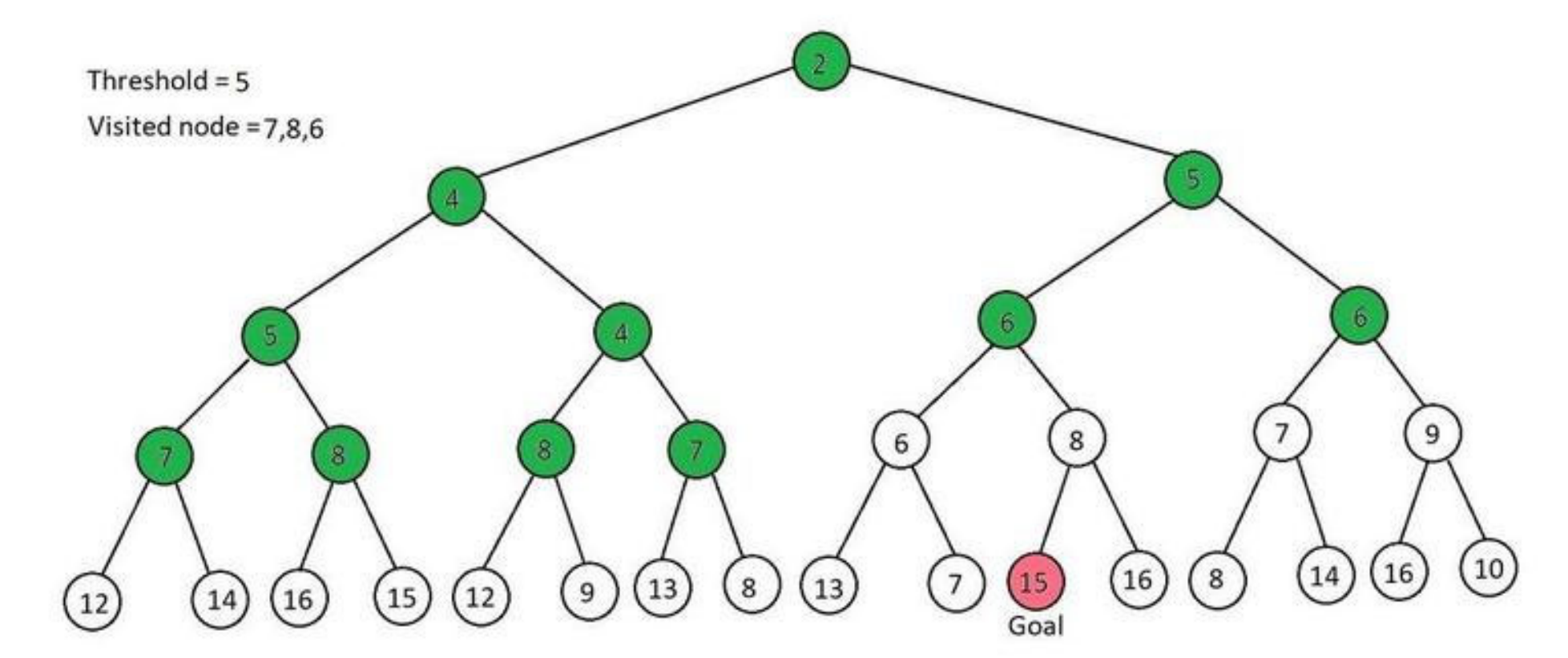
IDA* – Example: Iteration 1



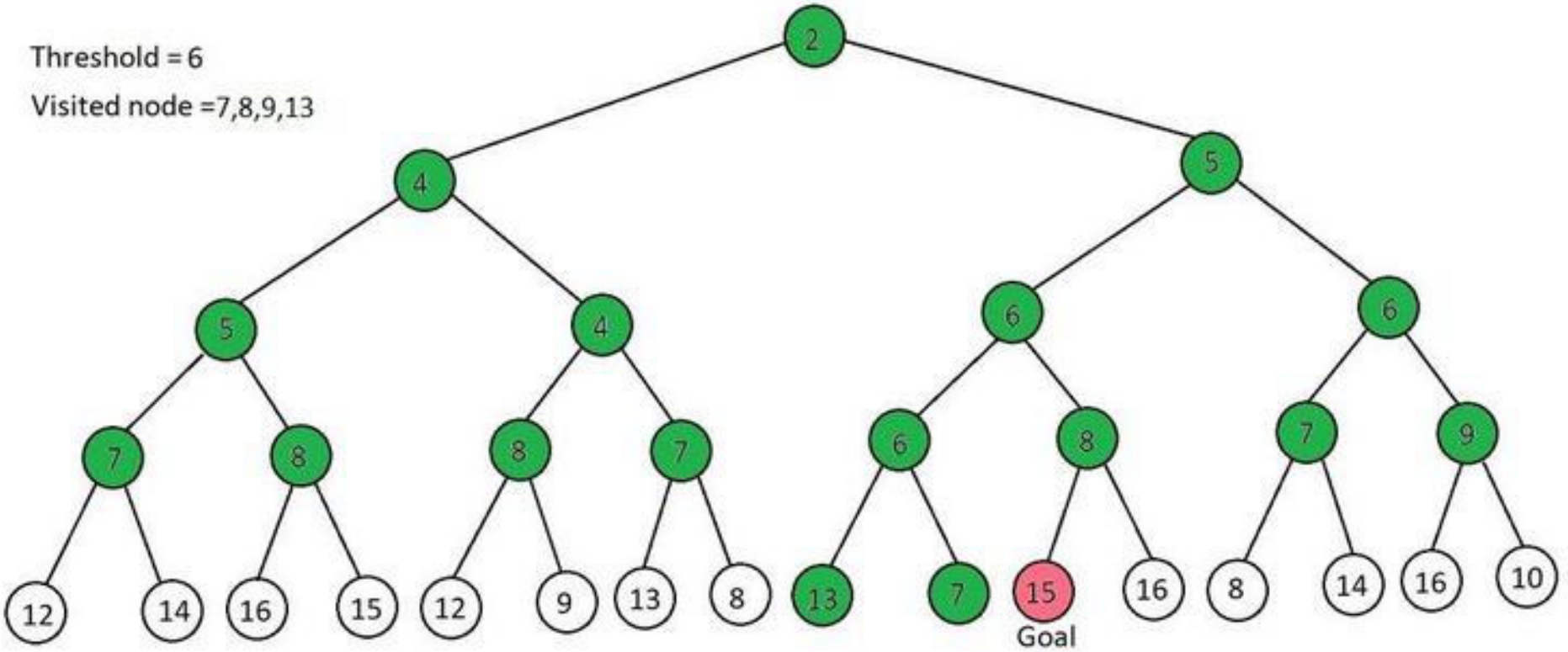
IDA* – Example: Iteration 2



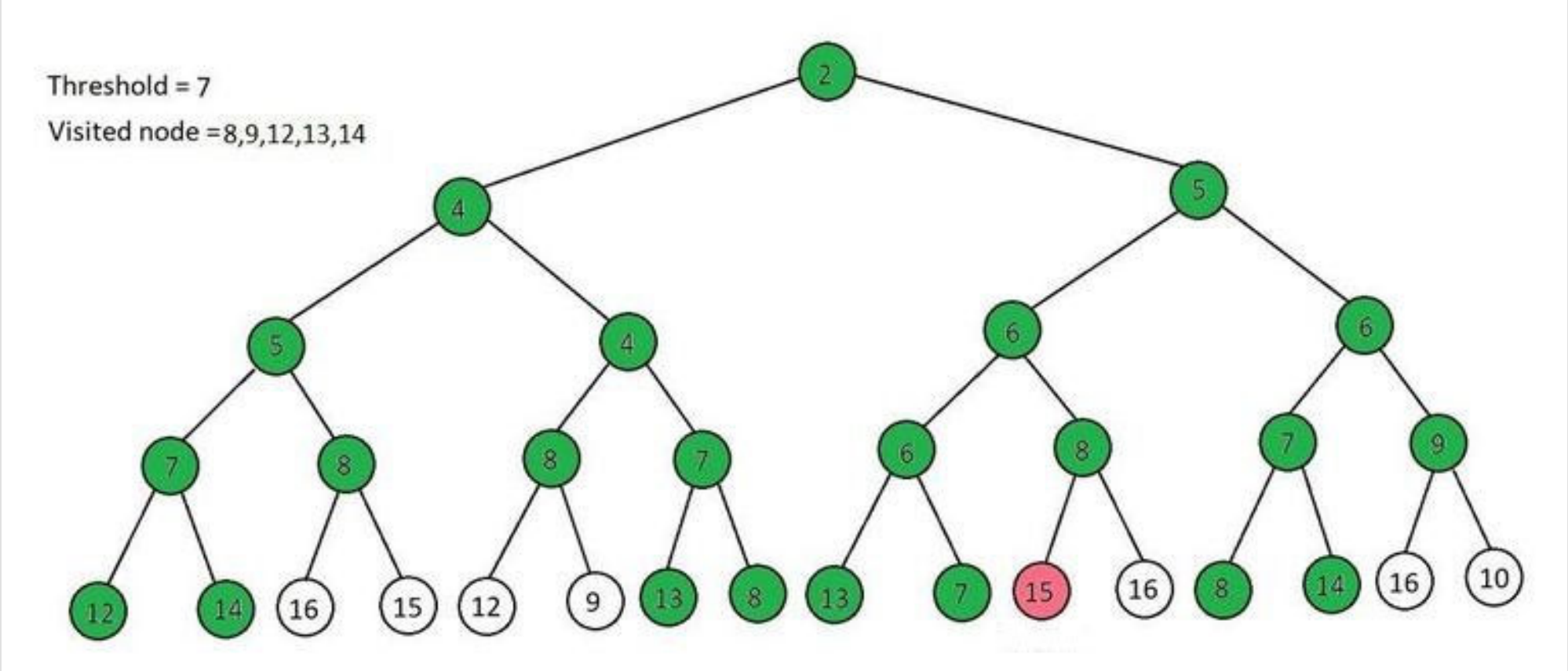
IDA* – Example: Iteration 3



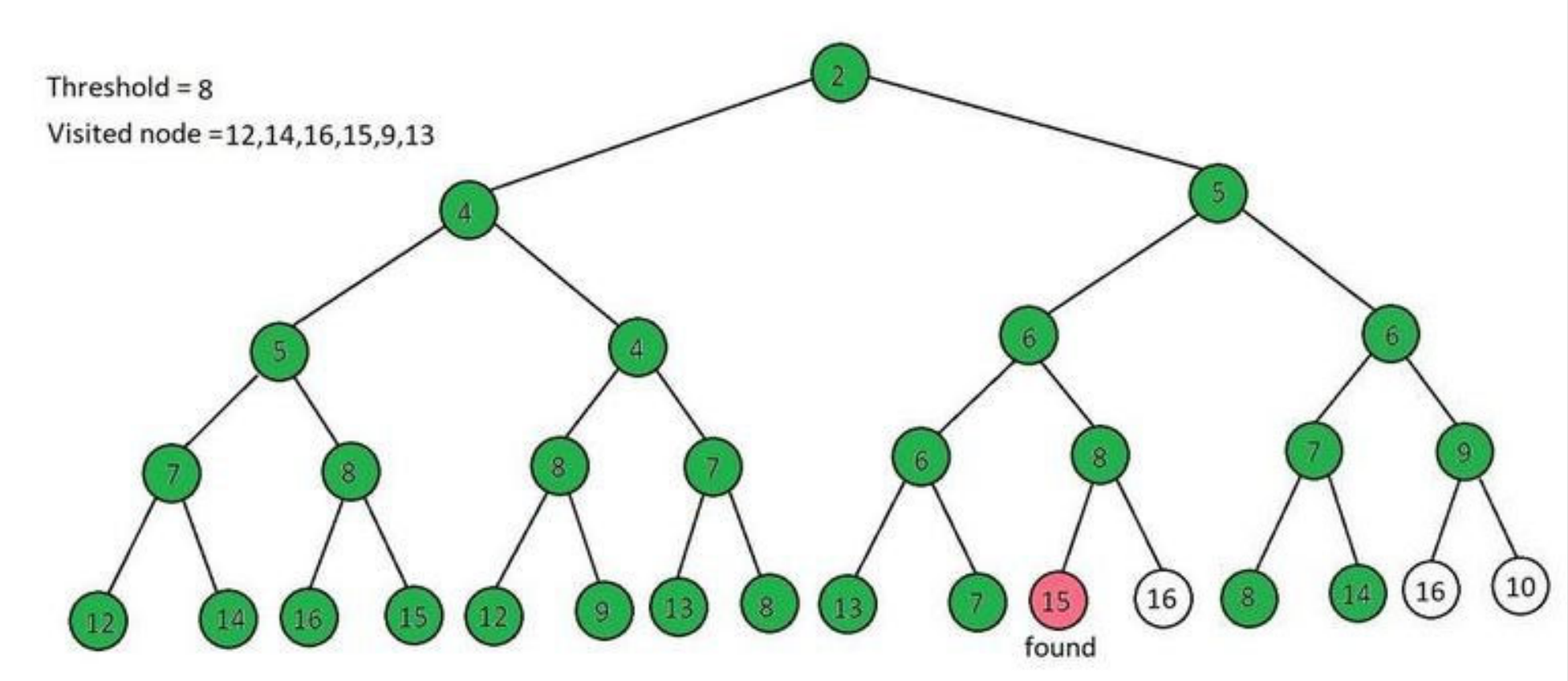
IDA* – Example: Iteration 4



IDA* – Example: Iteration 5



IDA* – Example: Iteration 6



Properties of IDA*

	Best case	Worst case
Complete?	Yes	Yes
Time?	*O(N), where N is the number of nodes expanded by A**	$O(N^2)$
Space?	*O(bd)*	*O(bd)*
Optimal?	Yes	Yes

Korf, Richard E. (1985). Depth-first Iterative-Deepening: An Optimal Admissible Tree Search. *Artificial Intelligence*. **27**: 97-109. doi:10.1016/0004-3702(85)90084-0. S2CID 10956233.

Weighted and Anytime Weighted A*

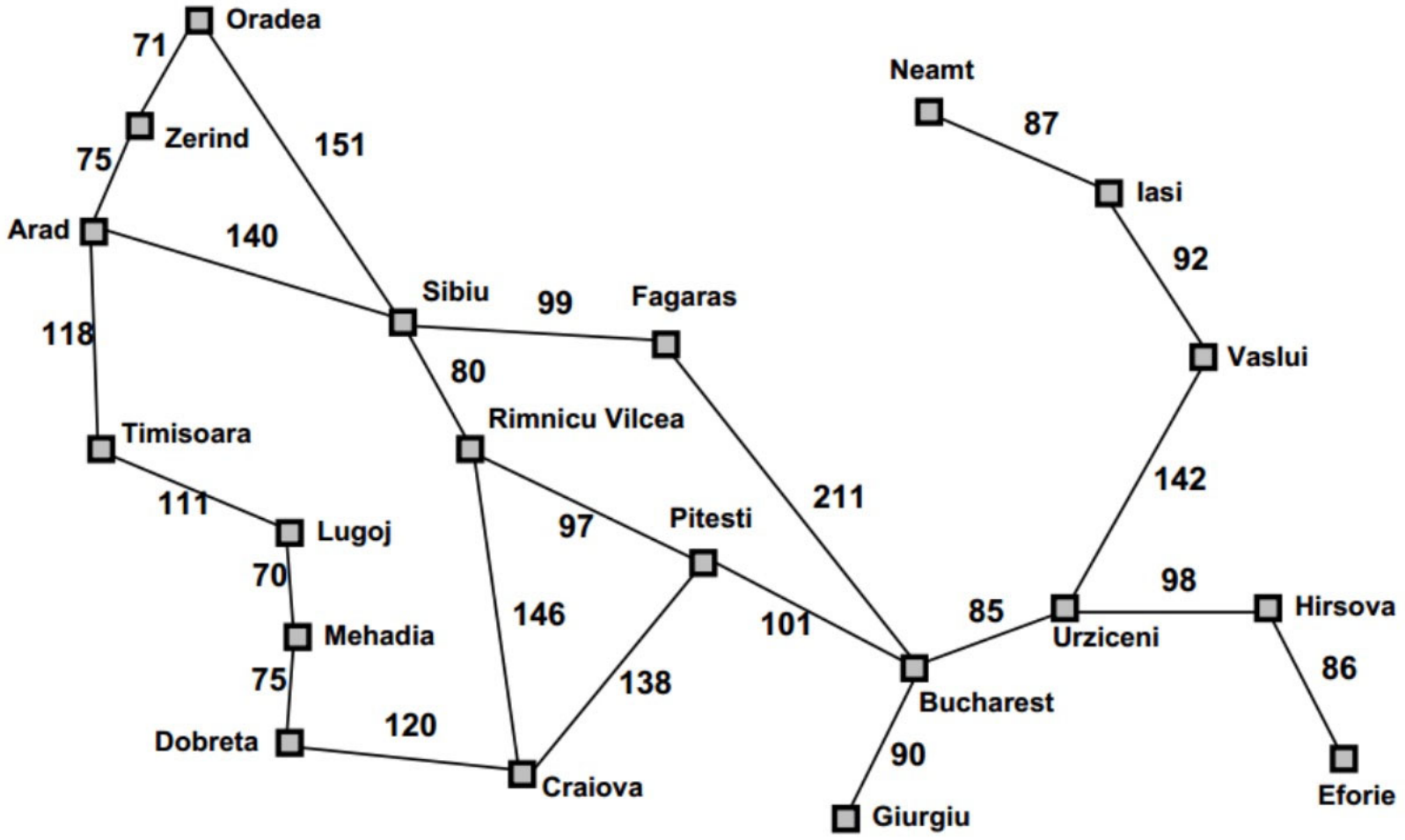
- **WA* – weighted A***
  - $f(n) = g(n) + w \times h(n), \qquad \text{with } w \geq 1$
  - Resultant heuristic may be inadmissible or inconsistent
  - Biased towards states that are closer to the goal
- **Anytime weighted A***
  - Continue searching after finding a solution, up to some time limit

Hansen, E.A. and Zhou, R., 2007. Anytime heuristic search. Journal of Artificial Intelligence Research, 28, pp. 267-297.

Properties of Weighted A*

	Like Algorithm A
Complete?	Yes
Time?	$O(b^m)$
Space?	$O(b^d)$
Optimal?	No

Practice Example – Romania Map

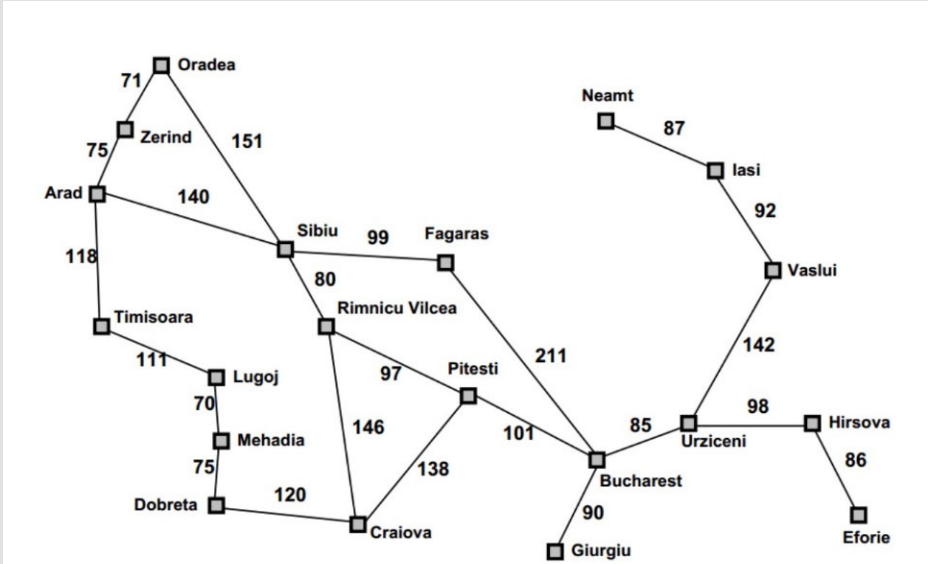


Practice example – Path Finding

- **Find a path between Arad and Pitesti**
  - For DFS and BFS: Alphabetical order
  - For A*: Straight-Line Distance (SLD), given below
  - For Greedy best-first search: SLD

Arad	260	**Mehadia**	140
Bucharest	100	**Neamt**	280
Craiova	110	**Oradea**	280
Drobeta	140	**Pitesti**	0
Eforie	210	**Rimnicu Vilcea**	90
Fagaras	100	**Sibiu**	150
Giurgiu	150	**Timisoara**	230
Hirsova	200	**Urziceni**	130
Iasi	270	**Vaslui**	250
Lugoj	140	**Zerid**	270

Practice Example – A* Solution



Arad	260	**Mehadia**	140
Bucharest	100	**Neamt**	280
Craiova	110	**Oradea**	280
Drobeta	140	**Pitesti**	0
Eforie	210	**Rimnicu Vilcea**	90
Fagaras	100	**Sibiu**	150
Giurgiu	150	**Timisoara**	230
Hirsova	200	**Urziceni**	130
Iasi	270	**Vaslui**	250
Lugoj	140	**Zerid**	270

Arad to Pitesti

I **Arad** h=260

Zer g= 75 h=270 f =345

II **Sib** g=140 h=150 f =290

Tim g=118 h=230 f =348

Ora g=291 h=280 f=571

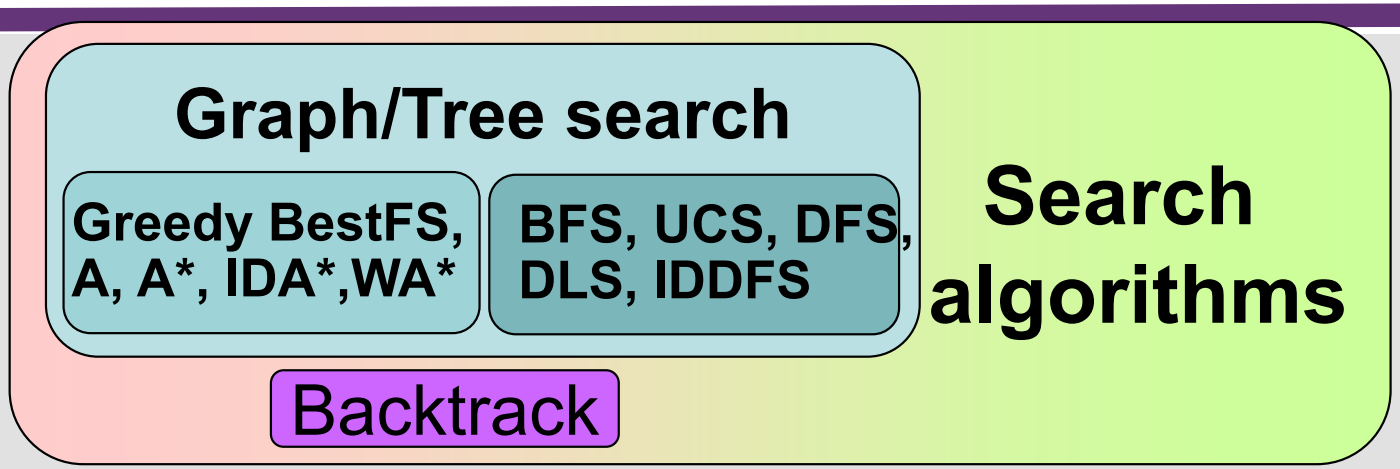
Fag g =239 h=100 f =339

III **RimVi** g=220 h= 90 f=310

Cra g=366 h=110 f=476

IV **Pit** g=317 h=0 f=317

Search Algorithms – A Perspective



- **Informedness in Graphsearch depends on g and h**
  - A $f(n) = g(n) + h(n)$ $(g(n) \geq g^*(n),\ 0 \leq h(n))$
  - A* $(g(n) \geq g^*(n),\ 0 \leq h(n) \leq h^*(n))$
- **Uninformed Graphsearch**
  - BFS ϵ A* when $g(n) =$ depth and $h(n) = 0$
  - UCS ϵ A* with $g(n) \geq 0$ and $h(n) = 0$
  - DFS, DLS, IDDFS ϵ Graphsearch, DFS, DLS, IDDFS $\not\epsilon$ A
- **Informed Graphsearch**
  - Greedy best-first search with $g(n) = 0$ and $h(n) \geq 0$ $\not\epsilon$ A

Summary: Treesearch and Graphsearch

- **When an agent is not clear on which immediate action is best, it can consider possible sequences of actions: search**
- **Before solutions can be found, the agent must formulate a goal and a problem, which consist of:**
  - the initial state; a set of operators; a set of constraints; a goal test function; a path cost function
- **A single general search algorithm can be used to solve any search problem**
- **Different search strategies yield different algorithms, which are judged on the basis of:**
  - completeness; optimality; time complexity; space complexity

Reading

- **Russell, S. and Norvig, P., *Artificial Intelligence – A Modern Approach* (4th ed), Prentice Hall**
  - Sections 3.5.1-3.5.5, 3.6.1-3.6.2

Next Lecture Topic

- **Lecture Topic 3c**
  - Problem Solving as Search – Irrevocable and Adversarial Search